

BrainLift — The Long Run

Owners

- **William** — primary owner / author
-

Purpose

Core Goal (North Star): To prove that the most effective way to teach probability is to make the learner **commit to a wrong answer before they're allowed to see the right one** — a *predict-then-verify* loop in which an interactive simulation is the **answer key**, not decoration. The Long Run is the existence proof: one subject taught deeply, where every gradable moment forces a numeric prediction, the math runs, and the gap between intuition and reality becomes the lesson. AI is layered on top only to *explain* numbers a deterministic engine already owns — never to compute them.

This BrainLift exists to make that thesis legible: *why* predict-then-verify, *why* withhold the answer, *why* keep the LLM away from the arithmetic, and *why* one subject instead of a catalog.

In Scope

- The pedagogy of **predict-then-verify** and the design choices that flow from it (withholding the answer, two-attempt grading, generated-not-fixed problems, "grade the decision, not the luck" in the Arcade).
- The **learning science** that justifies the friction the app deliberately adds (retrieval practice, desirable difficulties, productive failure, the generation effect, mastery learning, spaced/interleaved review).
- The **AI architecture stance**: deterministic source of truth (`probability.ts`, `blackjack.ts`) + an *optional, additive* LLM narrator behind a server-side Worker.
- **Probability as a domain** — specifically *why* it is the ideal subject for this format (the large gap between intuition and math; simulations that surprise).

- Motivation design that resists vanity metrics (streak earned only by the Problem of the Day; "feedback that explains, not grades").

Out of Scope

- A general theory of all education, or multi-subject breadth. The Long Run is deliberately **one subject**; this document does not argue for a content marketplace.
 - UI/visual-design system specifics (covered by the project's design skills and `index.css`), except where a UX choice is a pedagogical choice (e.g. hiding the sim during predict).
 - Letting AI **decide** anything (answers, EVs, grades). That path is rejected on principle, not explored.
 - K-12 / classroom deployment, teacher dashboards, real-money gambling, multiplayer — all explicitly out of scope for the product, so out of scope here.
 - Using this document to have an AI *generate* the insights for me. A BrainLift is structured context that passed through a human brain first.
-

DOK 4 — Spiky Points of View (SPOVs)

*Each SPOV is a strong, defensible, somewhat-contrarian stance, followed by an elaboration, the concrete way it's already **built** into the repo, and its research **grounding**.*

SPOV 1 — The simulation must be the *answer key*, not the illustration.

Almost every "interactive" learning product shows you the concept, then plays an animation to *illustrate* it. The animation is decoration: it arrives after you already know the answer, so it teaches nothing. The Long Run inverts the order. You are forced to **commit a numeric prediction first**; only then does the simulation run, and the simulation is what *grades* you. The decorative animation becomes a live assessment, and the moment of being wrong — watching 1,000 coin flips refuse to match your guess — is the actual instruction.

Elaboration: A prediction you generated yourself is encoded far more durably than a fact you read, and an error made with conviction is corrected more reliably than a fact you were simply told. By making the prediction mandatory and the

reveal contingent on it, every problem becomes a generation event *and* a retrieval event at once. The simulation isn't "engagement"; it's the ground truth the learner's intuition collides with.

In the build: `LessonPlayer` runs a `predict → run → feedback` loop on every gradable step; the sim's target lines / theoretical curves stay hidden until the run starts (`mode === 'explore' || processed > 0`) so they can't leak the answer. PRD **FR-1.6** makes predict-then-verify the *standard* gradable interaction, not an option.

Grounding: the [generation effect](#) (Slamecka & Graf, 1978); the **Predict–Observe–Explain** technique from science education (White & Gunstone, 1992); the [hypercorrection effect](#) (Butterfield & Metcalfe, 2001).

SPOV 2 — Withholding the answer *and* the simulation after a wrong first guess is a feature, not cruelty.

Conventional product UX says: never frustrate the user, reveal help immediately, make the correct path frictionless. For *learning*, that instinct is actively harmful. The Long Run does the opposite: a wrong **first** attempt reveals *nothing* — not the answer, not the explanation, and crucially **not even the simulation** — and silently deals a *different* generated version of the problem for the second try. The answer and the sim only appear once you've earned them (a correct answer) or exhausted your attempts (the second miss).

Elaboration: Immediate reveal short-circuits retrieval — the single most powerful lever for durable memory. The friction here is *desirable difficulty*: it slows the session down on purpose, in the exact spot where slowing down builds memory. Letting the learner see the sim after one wrong guess would convert a retrieval opportunity into passive recognition. The cost (a moment of productive struggle) is paid back as retention.

In the build: two-attempt grading (`green` first-try, `yellow` second-try, `red` twice wrong) in `store/progress.ts`; "answer withheld until resolved" is a documented hard rule; the second try pulls a freshly generated problem via the `retry` cycle index.

Grounding: the [testing effect](#) — Roediger & Karpicke (2006) found tested learners recalled ~56% after a week vs ~42% for re-readers, *reversing* the 5-minute result where re-reading led; [desirable difficulties](#) (Bjork & Bjork); [productive failure](#) (Kapur, 2008), where students who struggled *before* instruction beat directly-taught peers on transfer.

SPOV 3 — The LLM must never compute a single answer, EV, or grade. Its only job is to narrate numbers the deterministic engine already owns.

The industry reflex in 2026 is to put the language model at the *center* — "AI tutor," "AI grader," "ask the AI." For a domain with crisp, checkable ground truth, that is exactly backwards. An LLM doing arithmetic will eventually be *confidently* wrong, and in education a confidently-wrong answer key is the worst possible failure: it teaches the mistake. The Long Run draws a hard line — a deterministic core computes **every** number, and the model is a *narrator* that varies wording around those numbers.

Elaboration: This is a two-system design on purpose. System one (`probability.ts` , `blackjack.ts`) is boring, tested, and always right. System two (the LLM) is fluent, optional, and never trusted with a fact. The wrong-answer explainer is *handed* the app-computed correct answer as ground truth and asked only to explain *why*; the dealer-coach is *handed* the EVs and optimal action and asked only to phrase them. Unplug the AI entirely and the app is still whole — falling back to hand-written feedback and a templated coach. AI is **additive, never load-bearing**.

In the build: the OpenAI key lives in a free **Cloudflare Worker** (`worker/`), never the client bundle; `coachClient.ts` POSTs `{ kind, input } ( 'explain' / 'blackjack' )`. `explain.ts` ships the *computed* answer as truth; `blackjack.test.ts` Monte-Carlo-cross-checks the engine's EVs against textbook basic strategy. PRD **FR-10.2** / **NFR-7** codify "answers computed by the app, full functionality with AI off."

Grounding: the [survey of hallucination in NLG](#) (Ji et al., 2023) — fluency does not imply factuality; and the program-aided / tool-use result that offloading exact computation to deterministic code beats letting the model do the math — [PAL: Program-Aided Language Models](#) (Gao et al., 2023).

SPOV 4 — One subject taught deeply beats a catalog of everything.

Brilliant's brand *is* breadth — math, science, CS, dozens of courses. The Long Run deliberately refuses that. It teaches **probability and statistics only**: 36 lessons across 8 sections, plus a Final Test and an applied Arcade, all orbiting a single mental model — *what happens "in the long run."* Depth is the strategy, not a limitation.

Elaboration: A single deep spine lets every artifact reinforce the *same* ideas. Long-run frequency in Lesson 1 is the same intuition that makes the Galton board's bell curve inevitable, that defines expected value, that explains why the house edge is unbeatable at blackjack over 10,000 hands. A breadth catalog can't do this — its lessons are islands. Coherence compounds: by the Arcade, the learner isn't meeting EV for the first time, they're *applying* a concept the whole course has been building toward. The product name is the thesis.

In the build: `content/sections.ts` defines the 8-unit spine; the Arcade's "Blackjack Edge" sim auto-plays tens of thousands of perfect hands so the ~1% house edge becomes *visible* as cumulative net drifts below zero — the literal payoff of "the long run." Mastery-gated sequential unlock keeps the spine intact.

Grounding: [mastery learning](#) and Bloom's [2-sigma problem](#) (Bloom, 1984); the active-learning meta-analysis [Freeman et al. \(2014, PNAS\)](#) (active learning cut STEM failure rates and raised exam scores ~0.47 SD); the [PhET](#) lineage of deeply-designed interactive sims per concept.

SPOV 5 — Generated-from-templates problems beat fixed problem banks, because mastery means surviving *variation*, not recognizing an instance.

If a learner can memorize "the answer is 7," you have taught trivia, not probability. A fixed bank invites exactly this: enough repetition and the learner pattern-matches the *question* to a stored *answer* without ever running the reasoning. The Long Run generates each problem from a parameterized template that **recomputes the answer from** `probability.ts` every time, and *cycles* to new numbers/objects on retry and replay.

Elaboration: Varying the surface (numbers, scenarios, and even the *form* — a probability one time, "about how many of N..." the next) forces the learner to abstract the underlying schema instead of caching a result. The count-vs-probability alternation isn't cosmetic: natural-frequency phrasing makes probabilistic (especially Bayesian) reasoning dramatically more accessible, so alternating trains both representations. Because the answer is always derived programmatically, variation never risks a wrong key.

In the build: `content/problemTemplates.ts` — each slot's `build(rng)` returns a concrete step with the answer recomputed from the source-of-truth functions; generation is seeded by `seed × attempt × retry` so resume reproduces a question while retry/replay *cycle* to a different one. Numeric problems alternate probability/fraction and **count** phrasings.

Grounding: [variability of practice](#) / schema theory (Schmidt, 1975); retrieval-vs-recognition (PRD **FR-11.5**, "recall a value, not recognize from options"); natural frequencies — [Gigerenzer & Hoffrage \(1995\)](#) raised correct Bayesian inferences from ~16% to ~46% just by switching the *format* to counts.

SPOV 6 — In a game of chance, grade the *decision*, not the *outcome*.

Every casino game, and most of life, mixes skill with luck — and humans are terrible at separating them. We praise the player who hit on 16 and got a 5; we blame the one who stood on 18 and lost to a dealer 20. The Long Run's Arcade refuses to grade luck. It grades whether your action was **EV-optimal given the shoe**, tracking a lifetime *decision accuracy* that is invariant to whether any individual hand won.

Elaboration: This is predict-then-verify applied to *decisions*. You choose; the engine then reveals the EV-optimal action and the EV gap. Over one hand, optimal play can lose — that's variance, and it's the point. The companion "Blackjack Edge" sim auto-plays perfect strategy for tens of thousands of hands so the learner *sees* skill (decision accuracy) and luck (a single noisy hand) pull apart, and watches the negative house edge emerge only "in the long run." Separating signal from noise is arguably the single most valuable thing a probability course can teach.

In the build: `blackjack.ts` computes exact EV of hit/stand/double from live shoe composition (recursive DP), `argmax` optimal action, and Hi-Lo count; `BlackjackTable` shows decision accuracy + a cumulative-net chart; bankroll is play-chips only, framed "for learning, not gambling" (PRD **FR-9.3a/b**, **FR-9.3g**).

Grounding: the [law of large numbers](#) made experiential; outcome bias and the [gambler's fallacy](#) from the heuristics-and-biases program (Tversky & Kahneman, 1974).

SPOV 7 — Don't let the streak be bought with grinding. Protect the habit metric from the learning metric.

Standard gamification ties the streak to "do *any* activity today," which quietly turns the streak into something you *farm* — and turns lessons into streak fuel rather than learning. The Long Run decouples them on purpose: the **daily streak advances only by solving the Problem of the Day**. Lessons and the Final Test feed the *activity heatmap* (a record of effort) but never move the streak.

Elaboration: A streak should mean one specific, honest thing — "you showed up and did the daily reasoning rep" — not "you opened the app." Collapsing every action into one counter corrupts the signal and invites extrinsic-reward gaming, which research says can *crowd out* the intrinsic motivation (the satisfaction of mastery) you actually want to cultivate. Keeping the streak scarce, and keeping lesson feedback explanatory rather than score-y, supports competence and autonomy instead of point-chasing.

In the build: `recordActiveDay` is called *only* by the Problem of the Day (same day = no-op, next day = +1, gap = reset); the activity heatmap tallies *every* resolved problem separately; feedback "explains, not grades" (README persona; "feedback that explains, not just grades").

Grounding: [Self-Determination Theory](#) (Deci & Ryan) — autonomy/competence/relatedness drive durable motivation; the [overjustification effect](#) and [Deci, Koestner & Ryan \(1999\)](#), whose meta-analysis found tangible extrinsic rewards undermine intrinsic motivation.

Experts

The learning scientist whose work grounds the SPOVs above. Following him is how the thesis stays honest.

Carl Hendrick

- **Who:** Professor of Learning Science at Academica University of Applied Sciences (Amsterdam); PhD in education from King's College London; taught English in an inner-city London secondary school for 18 years before turning to research. Member of the UNESCO International Bureau of Education's Science of Learning editorial board.
- **Focus:** Bridging the gap between cognitive-science research and what actually happens in classrooms — retrieval practice, spacing, desirable difficulties, explicit instruction, and the "instructional illusions" that make ineffective teaching *feel* effective. Co-author of *How Learning Happens* and *How Teaching Happens* (with Paul Kirschner) and *Instructional Illusions*.
- **Why Follow:** He is the single best synthesizer of the exact research this BrainLift rests on — Bjork's desirable difficulties, Roediger & Karpicke's testing effect, Sweller's cognitive load, Kahneman's biases — translated into design decisions you can actually ship. His central warning is the throughline

of nearly every SPOV here: that fluency, engagement, and enjoyment are *poor proxies* for learning. Predict-then-verify, withholding the answer, and refusing to let a slick AI narration substitute for genuine retrieval are all attempts to optimize for real learning over the *illusion* of it. If a design choice in The Long Run can't survive his research-vs-practice test, it shouldn't ship.

- **Where:** <https://www.carlhendrick.com/> · [About](#) · [How Learning Happens](#) (Kirschner & Hendrick, 2020)

DOK 3 — Insights

Original conclusions synthesized from the sources below. Each connects to a source and to the SPOV(s) it supports.

From Retrieval, Generation & Productive Failure

Insight 1 — The reveal is the reward, so spend it carefully. The instant you show the answer (or the sim), the retrieval opportunity is gone forever. This reframes "showing the simulation" from a generous UX gift into a *scarce pedagogical currency*. The Long Run spends it only at the two moments it can't be wasted: a correct answer (reinforcement) or a second miss (resolution). *Source connection:* testing effect (Roediger & Karpicke). *SPOV connection:* SPOV 1, 2.

Insight 2 — A confident wrong prediction is worth more than a hesitant right one. The hypercorrection effect says high-confidence errors are corrected best. So the ideal design *raises* the learner's commitment before the reveal — a typed number, a dragged slider — because the bigger the surprise when the sim disagrees, the stickier the correction. *Source connection:* hypercorrection (Butterfield & Metcalfe); generation effect. *SPOV connection:* SPOV 1.

Insight 3 — "Failing" the first attempt should change the question, not just re-ask it. Re-asking the identical problem after a miss tests memory of *that instance*; dealing a freshly parameterized version tests the *concept*. Productive failure plus variability means the second try should never be the same numbers. *Source connection:* productive failure (Kapur); variability of practice. *SPOV connection:* SPOV 2, 5.

From Simulations & Cognitive Load

Insight 4 — A simulation only teaches if it can contradict you. A sim that plays after you know the answer can only confirm; a sim that plays after a *prediction* can *surprise*. The teaching power is entirely in the possibility of being proven wrong — which is why the predict step is non-negotiable and the target lines must stay hidden until the run. *Source connection:* PhET design research; Predict–Observe–Explain. *SPOV connection:* SPOV 1.

Insight 5 — "Explore freely" is only safe when the load is managed.

Unguided exploration can overwhelm working memory and degrade into floundering. The animate-small/batch-large rule (animate a handful of trials for intuition, compute the rest and animate only the aggregate) is really a cognitive-load decision dressed as a performance one — it keeps attention on the emerging pattern, not 1,000 moving dots. *Source connection:* Cognitive Load Theory (Sweller); Kirschner, Sweller & Clark. *SPOV connection:* SPOV 4 (guided depth), SPOV 1.

From Probabilistic Intuition & Representation

Insight 6 — The subject was chosen *because* intuition fails here.

Probability is not an arbitrary pick; it is the domain with the widest, most reliable gap between gut and math (Monty Hall, birthday problem, gambler's fallacy). That gap is the raw material of predict-then-verify — pick a domain where people are usually *right* and the loop has nothing to teach. *Source connection:* heuristics & biases (Tversky & Kahneman). *SPOV connection:* SPOV 1, 6.

Insight 7 — Switching between probability and counts trains two minds, not one. Because natural frequencies unlock reasoning that percentages hide, alternating "P(...)" with "about how many of N..." isn't just variety — it builds the translation skill *between* representations, which is where real statistical literacy lives. *Source connection:* natural frequencies (Gigerenzer & Hoffrage). *SPOV connection:* SPOV 5.

Insight 8 — Teaching someone to separate skill from luck is the highest-value lesson in the whole course. Most real-world "probability" failures are outcome bias — judging a decision by its result. The Arcade's "grade the decision, not the luck" is therefore not a gimmick bolted onto a game; it may be the most transferable thing the app teaches. *Source connection:* law of large numbers; outcome bias / gambler's fallacy. *SPOV connection:* SPOV 6.

From AI Architecture

Insight 9 — In a domain with ground truth, the LLM's value is

linguistic, not computational. The model's job is to turn a correct-but-terse number ("stand: +0.02 EV") into a sentence a human feels ("stand — the dealer's 6 busts ~42% of the time"). The moment you ask it to *produce* the 0.02, you've traded a guarantee for a vibe. *Source connection:* hallucination survey (Ji et al.); PAL (Gao et al.). *SPOV connection:* SPOV 3.

Insight 10 — "Works with AI off" is a forcing function for good

architecture. Requiring the app to be fully functional with no model and no backend forces a clean separation: deterministic core, then optional enhancement. That constraint is *why* the answer engine and the narrator can never tangle — the narrator literally isn't there half the time. *Source connection:* progressive enhancement; PRD NFR-7. *SPOV connection:* SPOV 3.

From Motivation & Mastery

Insight 11 — One counter can only honestly measure one thing. The instant a streak counts "any activity," it stops measuring habit and starts measuring app-opens. Splitting streak (Problem of the Day only) from the activity heatmap (everything) keeps each metric *true*, which is what lets them motivate without corrupting behavior. *Source connection:* SDT; overjustification (Deci, Koestner & Ryan). *SPOV connection:* SPOV 7.

Insight 12 — Mastery gating is what makes depth possible. A deep single spine only works if each rung is solid before the next; clear-before-unlock (and all-green-for- mastery) is the mechanism that lets later lessons *assume* earlier intuition — the compounding that breadth catalogs can't achieve. *Source connection:* mastery learning (Bloom). *SPOV connection:* SPOV 4.

DOK 2 — Knowledge Tree

1. Retrieval Practice, Generation & Desirable Difficulty

Summary: This category underpins the entire predict-then-verify mechanic and the decision to withhold answers. The throughline: the *act of trying to produce an answer from memory* — and being allowed to be wrong — is a more powerful learning event than any amount of clear explanation delivered up front.

1.1 The Testing Effect

- **Source:** Roediger, H. L., & Karpicke, J. D. (2006). *Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention*. *Psychological Science*, 17(3), 249–255.
 - **DOK 1 — Facts:**
 - On an immediate (5-min) test, re-studying beat testing (~81% vs ~75%).
 - The result **reversed** with delay: after one week, the tested group recalled ~56% vs ~42% for the re-study group — with no feedback given during testing.
 - In Experiment 1, proportional forgetting over a week was ~10% (repeated testing) vs ~52% (repeated study).
 - **DOK 2 — Summary:** Being tested is not just measurement; it *changes* memory. Short-term, re-reading looks better, which is exactly the trap — designers optimizing for the immediate session pick the worse long-term strategy. The Long Run optimizes for the delayed curve: every problem is a test, and the reveal is withheld so the retrieval actually happens.
 - **Link:** <https://journals.sagepub.com/doi/10.1111/j.1467-9280.2006.01693.x>

1.2 Desirable Difficulties

- **Source:** Bjork, R. A., & Bjork, E. L. *Making things hard on yourself, but in a good way: Creating desirable difficulties to enhance learning*.
 - **DOK 1 — Facts:**
 - Conditions that slow acquisition (spacing, interleaving, testing, reduced feedback) often *improve* long-term retention and transfer.
 - Learners' subjective sense of fluency is a poor and often inverted guide to actual learning.
 - **DOK 2 — Summary:** The fluency illusion is the enemy. Choices that make a session feel harder (hide the sim, change the retry problem) are the ones that build durable skill — but they must be *desirable* (productive), not arbitrary. This is the explicit license for the friction in SPOV 2.
 - **Link:** <https://bjorklab.psych.ucla.edu/research/>

1.3 The Generation Effect

- **Source:** Slamecka, N. J., & Graf, P. (1978). *The generation effect: Delineation of a phenomenon*. *Journal of Experimental Psychology: Human Learning and Memory*.
 - **DOK 1 — Facts:**

- Information a learner *generates* is retained better than identical information they merely *read*.
- The effect is robust across many materials and tasks.
- **DOK 2 — Summary:** A typed prediction is generated, not read — so the prediction step earns retention before the simulation even runs. This is the core reason a number box *before* the sim beats any amount of explanation after it.
- **Link:** https://en.wikipedia.org/wiki/Generation_effect

1.4 Productive Failure

- **Source:** Kapur, M. (2008). *Productive Failure*. Cognition and Instruction, 26(3), 379–424 (and the 2010 Singapore replication).
- **DOK 1 — Facts:**
 - Students who tackled complex problems *without* support first produced poor initial solutions but **outperformed** directly-instructed peers on later near- and far- transfer tests.
 - The 2010 follow-up (7th-grade math, Singapore) replicated the advantage over a lecture-and-practice design.
- **DOK 2 — Summary:** Struggle *before* instruction is productive, not wasteful. This is the justification for making the learner commit (and possibly fail) a prediction before any concept is "taught," and for re-surfacing the lecture only after two misses.
- **Link:** <https://www.tandfonline.com/doi/abs/10.1080/07370000802212669>

1.5 Hypercorrection of High-Confidence Errors

- **Source:** Butterfield, B., & Metcalfe, J. (2001); Metcalfe, J. (2017). *Learning from Errors*. Annual Review of Psychology, 68.
- **DOK 1 — Facts:**
 - Errors committed with *high confidence* are more likely to be corrected after feedback than low-confidence errors — the "hypercorrection effect."
 - Surprise at being wrong appears to drive attention to the correction.
- **DOK 2 — Summary:** Don't soften the commitment — sharpen it. A bold, confident prediction that the simulation contradicts produces the most durable correction, so the UI should encourage real commitment, not tentative hedging.
- **Link:** <https://www.annualreviews.org/doi/10.1146/annurev-psych-010416-044022>

2. Interactive Simulations & Cognitive Load

Summary: This category governs *how* the simulations are built so they teach instead of merely entertain, and keeps "explore freely" from violating working-memory limits.

2.1 Research-Based Interactive Simulations (PhET)

- **Source:** PhET Interactive Simulations research program (Wieman, Perkins, et al.), University of Colorado Boulder.
 - **DOK 1 — Facts:**
 - Well-designed sims with *implicit* scaffolding can produce learning gains comparable to or exceeding traditional instruction.
 - Productive exploration depends on careful constraint of what the learner can do and see at each moment.
 - **DOK 2 — Summary:** A simulation is a teaching instrument only if its affordances are designed — including *hiding* the answer until the right moment. The Long Run's rule that target lines/curves stay hidden until a run starts is a direct application.
 - **Link:** <https://phet.colorado.edu/en/research>

2.2 Cognitive Load Theory & the Limits of Discovery

- **Source:** Sweller, J. (1988); Kirschner, P. A., Sweller, J., & Clark, R. E. (2006). *Why Minimal Guidance During Instruction Does Not Work*. Educational Psychologist, 41(2), 75–86.
 - **DOK 1 — Facts:**
 - Working memory is sharply limited; novices overwhelmed by unguided tasks learn less.
 - Pure discovery/minimal-guidance instruction is consistently outperformed by guided approaches (worked examples, scaffolding).
 - **DOK 2 — Summary:** The nuance that keeps the product honest: The Long Run is *guided* discovery. The computed answer, the lecture, scaffolding, and animate-small/batch-large (never render thousands of sprites) all manage load so exploration stays productive.
 - **Link:** https://www.tandfonline.com/doi/abs/10.1207/s15326985ep4102_1

2.3 Active Learning Outperforms Lecture (STEM)

- **Source:** Freeman, S., et al. (2014). *Active learning increases student performance in science, engineering, and mathematics*. PNAS, 111(23), 8410–8415.
 - **DOK 1 — Facts:**

- Meta-analysis of 225 studies: active learning raised average exam scores by ~ 0.47 SD; lecture-based courses had $\sim 1.5\times$ the failure rate of active-learning courses.
- **DOK 2 — Summary:** The strongest aggregate evidence that "doing" beats "being told" in exactly this subject family — the empirical mandate for a learn-by-doing app over a video-lecture one.
- **Link:** <https://www.pnas.org/doi/10.1073/pnas.1319030111>

3. Probabilistic Reasoning & Representation

Summary: Justifies *why probability* and *why the count-vs-probability alternation* — the domain and the framing are chosen to exploit (and repair) known failures of intuition.

3.1 Heuristics & Biases

- **Source:** Tversky, A., & Kahneman, D. (1974). *Judgment under Uncertainty: Heuristics and Biases*. Science, 185(4157), 1124–1131.
- **DOK 1 — Facts:**
 - People systematically misjudge probability via representativeness, availability, and base-rate neglect.
 - Related work documents the gambler's fallacy and "belief in the law of small numbers."
- **DOK 2 — Summary:** Intuition is *reliably* wrong about probability — which is the precondition that makes predict-then-verify teach. The simulation has something to correct because the gut almost always mispredicts.
- **Link:** <https://www.science.org/doi/10.1126/science.185.4157.1124>

3.2 Natural Frequencies

- **Source:** Gigerenzer, G., & Hoffrage, U. (1995). *How to Improve Bayesian Reasoning Without Instruction: Frequency Formats*. Psychological Review, 102(4), 684–704.
- **DOK 1 — Facts:**
 - Reframing Bayesian problems from probabilities to natural frequencies raised correct inferences from $\sim 16\%$ to $\sim 46\%$ (up to $\sim 50\%$) — *without any teaching*.
 - The format effect was $\sim 3\times$ larger than other manipulations tested.
- **DOK 2 — Summary:** Representation is pedagogy. Alternating "about how many of N..." with "P(...)" gives learners the count format that unlocks

reasoning *and* trains translation between the two — the basis of SPOV 5's generated count/probability mix.

- **Link:** <https://psycnet.apa.org/doi/10.1037/0033-295X.102.4.684>

3.3 Law of Large Numbers, Experienced

- **Source:** Classical probability (the "long run") + outcome-bias literature.
 - **DOK 1 — Facts:**
 - Sample averages converge to the expected value as trials grow; short runs are noisy and misleading.
 - Humans routinely judge decisions by single outcomes (outcome bias) rather than expected value.
 - **DOK 2 — Summary:** The product's name and spine. Making the long run *visible* — 1,000 flips converging, tens of thousands of blackjack hands drifting below zero — is how the app turns an abstract theorem into something the learner has *seen*, and how the Arcade separates skill from luck (SPOV 6).
 - **Link:** https://en.wikipedia.org/wiki/Law_of_large_numbers

4. AI as Narrator, Not Oracle

Summary: The architectural backbone of SPOV 3 — keep a deterministic source of truth and let the LLM only phrase it.

4.1 LLM Hallucination & Calibration

- **Source:** Ji, Z., et al. (2023). *Survey of Hallucination in Natural Language Generation*. ACM Computing Surveys, 55(12).
 - **DOK 1 — Facts:**
 - LLMs produce fluent, confident output that is unfaithful to source or fact — "hallucination" — as a systemic property, not a rare bug.
 - Fluency and correctness are decoupled; confident phrasing is not evidence of truth.
 - **DOK 2 — Summary:** In a domain with checkable answers, trusting the model with the number is an unforced error. The Long Run hands the model the *already-correct* number and asks only for words — so a hallucinated value can never reach the learner.
 - **Link:** <https://dl.acm.org/doi/10.1145/3571730>

4.2 Program-Aided / Tool-Use Grounding

- **Source:** Gao, L., et al. (2023). *PAL: Program-Aided Language Models*.

- **DOK 1 — Facts:**
 - Offloading computation from the LLM to executed (deterministic) code substantially improves accuracy on quantitative tasks.
 - The model is better used to *decompose* / *phrase* than to *calculate*.
- **DOK 2 — Summary:** The research-backed pattern The Long Run already follows: `probability.ts` / `blackjack.ts` are the "program," the LLM is the language layer. This is the principled version of "AI varies wording, the app owns the numbers."
- **Link:** <https://arxiv.org/abs/2211.10435>

5. Motivation, Mastery & Habit

Summary: Why the streak is scarce, why feedback explains rather than grades, and why mastery gating enables depth.

5.1 Self-Determination Theory & the Overjustification Effect

- **Source:** Deci, E. L., Koestner, R., & Ryan, R. M. (1999). *A meta-analytic review of experiments examining the effects of extrinsic rewards on intrinsic motivation*. Psychological Bulletin, 125(6), 627–668.
- **DOK 1 — Facts:**
 - Tangible extrinsic rewards, especially when expected and performance-independent, reliably *undermine* intrinsic motivation.
 - Intrinsic motivation is supported by autonomy, competence, and relatedness.
- **DOK 2 — Summary:** Over-gamifying (a streak you can farm by any action) risks crowding out the intrinsic satisfaction of understanding. Keeping the streak tied to one honest daily rep, and feedback explanatory rather than score-y, protects the real motivator (SPOV 7).
- **Link:** <https://psycnet.apa.org/doi/10.1037/0033-2909.125.6.627>

5.2 Mastery Learning

- **Source:** Bloom, B. S. (1984). *The 2 Sigma Problem*. Educational Researcher, 13(6), 4–16; Bloom (1968), *Learning for Mastery*.
- **DOK 1 — Facts:**
 - One-to-one tutoring with mastery learning moved the average student ~2 SD above conventional-class peers (to ~98th percentile).
 - Mastery learning requires demonstrated competence before advancing.
- **DOK 2 — Summary:** Clear-before-unlock and all-green-for-mastery are the scalable echo of this finding, and the mechanism that makes a deep

single-subject spine possible (SPOV 4): each lesson can assume the prior one is solid.

- **Link:** <https://journals.sagepub.com/doi/10.3102/0013189X013006004>
-

References

- Bjork, R. A., & Bjork, E. L. *Making things hard on yourself, but in a good way: Creating desirable difficulties to enhance learning.*
- Bloom, B. S. (1984). The 2 Sigma Problem. *Educational Researcher*, 13(6), 4–16.
- Butterfield, B., & Metcalfe, J. (2001). Errors committed with high confidence are hypercorrected. *JEP: LMC*, 27(6).
- Deci, E. L., Koestner, R., & Ryan, R. M. (1999). A meta-analytic review of experiments examining the effects of extrinsic rewards on intrinsic motivation. *Psych. Bulletin*, 125(6), 627–668.
- Freeman, S., et al. (2014). Active learning increases student performance in science, engineering, and mathematics. *PNAS*, 111(23), 8410–8415.
- Gao, L., et al. (2023). PAL: Program-Aided Language Models. *ICML*.
- Gigerenzer, G., & Hoffrage, U. (1995). How to improve Bayesian reasoning without instruction: Frequency formats. *Psychological Review*, 102(4), 684–704.
- Ji, Z., et al. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12).
- Kapur, M. (2008). Productive Failure. *Cognition and Instruction*, 26(3), 379–424.
- Kirschner, P. A., Sweller, J., & Clark, R. E. (2006). Why minimal guidance during instruction does not work. *Educational Psychologist*, 41(2), 75–86.
- Metcalfe, J. (2017). Learning from errors. *Annual Review of Psychology*, 68, 465–489.
- Roediger, H. L., & Karpicke, J. D. (2006). Test-enhanced learning. *Psychological Science*, 17(3), 249–255.
- Slamecka, N. J., & Graf, P. (1978). The generation effect. *JEP: Human Learning and Memory*, 4(6), 592–604.
- Tversky, A., & Kahneman, D. (1974). Judgment under uncertainty: Heuristics and biases. *Science*, 185(4157), 1124–1131.

- White, R., & Gunstone, R. (1992). *Probing Understanding* (Predict–Observe–Explain).
- Wieman, C., & Perkins, K., et al. PhET Interactive Simulations research.
<https://phet.colorado.edu/en/research>